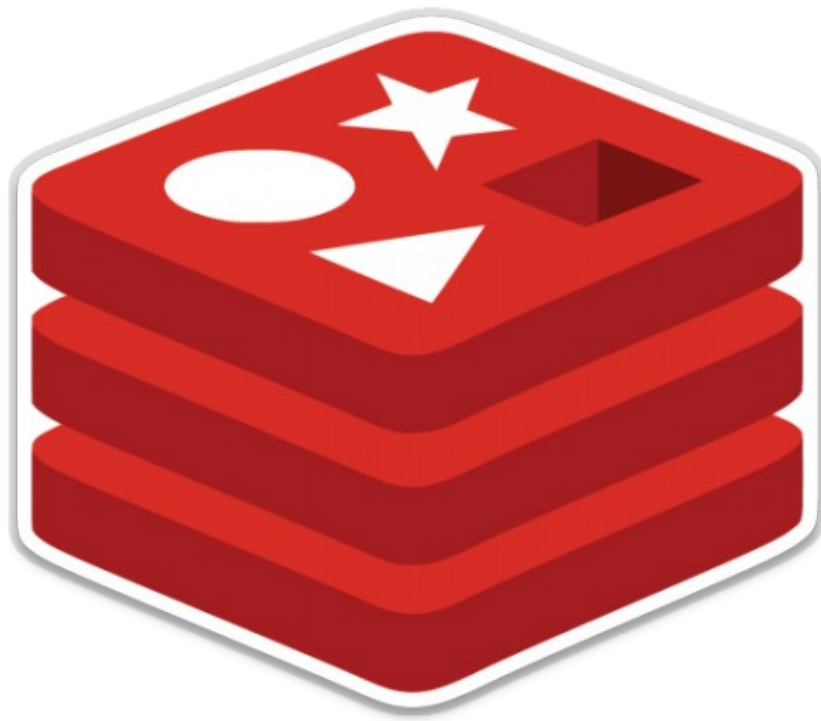


REDIS



Rapport sur l'analyse de la base de données Redis

Auteur : Vincent Roger

UE : NFE 204

Professeur : Philippe Rigaux

Table des matières

I. Introduction.....	3
II. Installation et mise en place d'un serveur Redis.....	4
1) Sur un serveur dédié sous Linux/Ubuntu.....	4
2) Sur un container de type docker.....	4
a) Installation du serveur :.....	4
b) Connexion sur le client docker :.....	4
III. Interagir avec le serveur.....	4
1) Outils d'interaction.....	4
2) Démarrage du serveur.....	4
3) Les clients.....	5
IV. Structures de données.....	5
1) Types string :.....	5
a) Commandes possibles à partir du type String.....	6
b) Redis expires.....	6
c) Redis list.....	7
d) Redis hashes.....	8
e) Redis sets (ensembles non triés).....	8
f) Redis sorted sets (ensembles triés).....	9
g) Redis Hyperloglogs.....	10
V. Base de données géospatiale.....	10
1) Fonctions d'insertion et de recherche.....	10
a) GEOADD.....	10
b) GEOHASH.....	11
c) GEOPOS.....	11
2) Opérateur de comparaison.....	11
a) GEODIST.....	12
b) GEORADIUS.....	12
c) GEORADIUSBYMEMBER.....	13
VI. Le système de sauvegardes.....	13
1) Créer une copie exacte d'un serveur.....	13
2) Persistance des données sur disques.....	14
a) RDB.....	14
b) AOF.....	15
c) Stratégie de sauvegarde.....	15
VII. Fonctionnement du mode cluster.....	15
1) Mise en place :.....	16
2) En cas de panne :.....	16
3) Répartition des clés ou resharding.....	18
VIII. Clients Redis pour les langages de programmation.....	18
1) Utilisation de la bibliothèque.....	18
2) Mode pipeline de Redis.....	19
IX. Mise en application dans un jeu de données.....	19
1) Code python.....	20
2) Récupération des valeurs.....	21
X. Conclusion.....	21
XI. Références bibliographique.....	21

I. Introduction

Redis est une base de données open-source sous licence BSD dit « in-memory data store » écrite en C dont il existe des connecteurs avec beaucoup de langages de programmation.

Redis est l'acronyme de « Remote Dictionary Server » qui signifie serveur de dictionnaire distant.

Redis utilise les principes de stockage « clé-valeur » avec un ensemble de règles et d'opérateurs pour améliorer les performances de fonctionnement sur de la mémoire RAM (sans délai de latence). Ce mode de fonctionnement avec les techniques d'optimisation en font une base de données extrêmement performante et très facile d'utilisation tant ses structures de données sont simples.

La rapidité d'une telle base de données en fait une solution extrêmement efficace dans les domaines du WEB, mobiles, jeux, l'internet des objets connectés qui demande un traitement ultra-rapide, ainsi que dans de nombreux autres domaines.

En comparaison avec ses concurrents, le temps de traitement (insertion et restitution) dans une base de données Redis offre des performances sans égal au sacrifice d'une hiérarchie de données comme on peut le voir sur des données dans le format JSON. Le fait de travailler sur de la mémoire RAM et non sur un disque dur permet de réduire les délais de latence. De plus, la structure étant simplifiée, les algorithmes sont plus simples, paraissent plus optimisés et demandent moins d'instruction CPU et sont donc moins gourmands.

Ce sera donc une base de données de choix dans le cloud, où les processeurs sont inférieurs en règle générale à ceux utilisant des serveurs dédiés.

Par contre, le stockage sur RAM restera bien plus onéreux. Il faudra donc utiliser cette base de données pour des données de tailles relativement réduite (bien qu'elle fonctionne en cluster avec répartition des données) et demandant une très grande réactivité, ou disposer de moyens financiers conséquents.

Redis fonctionne exclusivement avec des clés stockées dans des tables de hachage afin de répartir l'ensemble des données sur plusieurs serveurs si nécessaire et retrouver l'information de la manière la plus rapide possible.

Chaque clé est associée à une ou plusieurs valeurs dont la taille maximum est de 512Mo et prend en charge plusieurs types de données :

- Les chaînes de caractères
- Des ensemble de données triées ou non
- Des listes
- Des données géospatiales

Nous rentrerons plus en détails dans la suite de ce rapport.

Redis prend en charge un système de réplication de données sur plusieurs serveurs qui fonctionnent en mode maître-esclave exclusivement. Redis propose également un fonctionnement en mode dit « cluster » qui permet la répartition des données par utilisation de la structure de stockage des clés sur une table de hachage.

Pour assurer un certains degré de durabilité, Redis dispose également d'outils de persistance de données pour le stockage sur disque divisé en 2 catégories :

Fichier AOF (Append Only File) qui est un dictionnaire des changements de valeurs

Fichier RDB qui est une image de la base à un instant donné.

II. Installation et mise en place d'un serveur Redis

1) Sur un serveur dédié sous Linux/Ubuntu

L'installation recommandée par Redis est la compilation des sources d'après la dernière version stable.

→ ne nécessite pas dépendance particulière, la compilation se réalise avec GCC

```
wget http://download.redis.io/redis-stable.tar.gz
tar xvzf redis-stable.tar.gz
cd redis-stable
make OU sudo make install
→ permet d'installer les commandes de bases
```

2) Sur un container de type docker

a) Installation du serveur :

```
docker pull redis
docker run -p 6379:6379 --rm --name mon-redis redis
```

b) Connexion sur le client docker :

```
docker exec -ti mon-redis redis-cli (-h <host_name> -p <port>) par défaut sans arguments h ou p, redis prend localhost et le port par défaut.
```

III. Interagir avec le serveur

1) Outils d'interaction

- **redis-server** : permet de lancer le serveur
- **redis-sentinel** : permet le monitoring de l'état de santé du serveur et de vérifier l'état de santé des failovers
- **redis-cli** : client en ligne de commandes pour interroger la base de données
- **redis-benchmark** : permet de vérifier les performances de son serveur
- **redis-check-aof** et **redis-check-rdb** : outils d'administration utiles en cas de corruption de données

2) Démarrage du serveur

Redis propose par défaut une configuration sans réplication, sans persistance de données et sans sécurité d'authentification écoutant le port par défaut 6379 sur la machine locale.

Pour un serveur en production, il faudra donc modifier ces paramètres de base dans un fichier de configuration se trouvant dans les sources de Redis se nommant « redis.conf ».

Ce fichier est largement documenté et permet d'activer toutes les fonctionnalités intéressantes que l'on attend d'une base de données distante.

Il y sera donc possible de déterminer la stratégie de réplication/cluster, la stratégie de persistance des données. Nous montrerons au fur et à mesure des besoins des exemples de commandes à utiliser et ajouter dans ce fichier.

```
> redis-server
> redis-server etc/redis.conf (permet d'associer un fichier de configuration
spécifique au serveur)
> redis-cli ping (le client envoie un ping et un pong est retourné si le
serveur répond bien)
PONG
```

Ports utilisés :

- Port 6379 : port d'écoute pour les requêtes, par défaut, le serveur n'écoute qu'en localhost
- Port 16379 : utilisé par les clusters (+10000)
- Port 26379 : utilisé pour redis-sentinel (+20000)

Protection du serveur avec mot de passe :

Il faut introduire la commande « requirepass » avec un mot de passe dans le fichier de configuration et démarrer le serveur comme cité précédemment.

Lors de la connexion au serveur avec le client en ligne de commandes, taper :

```
AUTH <password>
AUTH <user> <password>
```

La liste des commandes de gestion des utilisateurs se trouve sur la documentation officielle : <https://redis.io/commands/auth>

3) Les clients

Par défaut, Redis propose un client en ligne de commande qui suffit à comprendre et interagir avec la base de données. Ce client sera privilégié tout au long de ce rapport mais il est possible également de bénéficier d'un client graphique comme Redis-commander qui est une application node.js et open-source ou Redsmin qui est une solution payante et utilisable aisément sur un système de type cloud.

Il existe aussi Redis Desktop Manager et quelques autres. Ces interfaces offrent dans la majorité des cas l'inspection d'une base et la connexion sur plusieurs bases simultanément.

Pour la suite de ce rapport, nous utiliserons exclusivement le client Redis officiel par défaut avec les sources Redis.

```
> redis-cli -h <host> -p <port>
```

IV. Structures de données

1) Types string :

Le type string (chaîne de caractères) est la plus simple des valeurs, elle s'associe à une clé et peut être définie de cette manière :

```
> set clé valeur (pour l'insertion)
> get clé (pour la récupération)
```

Il est également possible d'insérer une image qui sera enregistrée sous forme de string, il subsiste tout de même une limitation, ce champs ne peut excéder 512MB.

Si la clé existe déjà, sa valeur sera remplacée ou insérée si elle n'existe pas.

Il est possible également de ne pas insérer la valeur si la clé existe déjà avec un paramètre **nx** ou **xx**

```
> set clé nouvellevaleur nx
(nil)
> set clé nouvellevaleur xx
OK
```

a) Commandes possibles à partir du type String

- **INCR** : cette commande transforme le type string en integer et incrémente sa valeur

```
> set compteur 100
> incr compteur
(integer) 101
```

- **decr** : commande similaire mais décrémente la valeur par un
- **incrby** : commande similaire mais ajoute une valeur d'incrément
- **decrby** : commande similaire mais ajoute une valeur de décrémentation

→ **attention : il s'agit de valeurs atomiques, cela signifie que si deux commandes incr parviennent simultanément, la valeur est incrémentée de deux.**

- **GETSET** : permet la modification d'une clé.
- **MSET** et **MGET** : permettent de gérer un ensemble de clés (insertion multiple)

```
> MSET a 1 b 2 c 3
ok
> MGET a b c (Retourne un tableau de valeurs)
1) « 1 »
2) « 2 »
3) « 3 »
```

- **exists** : teste l'existence d'une clé
- **del** : supprime une clé et sa valeur

```
> exists clé
> del clé
```

→ **valeur de retour 1 ou 0 en fonction de la possibilité de l'action**

b) Redis expires

Une des particularités de Redis est de pouvoir ajouter une notion de TTL sur les clés, cela signifie qu'il est possible pour la base de supprimer une clé et sa valeur au bout d'un certain délai exprimé en secondes ou en milli-secondes. La gestion des données en ram étant limitée en terme d'espace, cette notion de TTL s'avère particulièrement pratique en lieu et place de nettoyer les données automatiquement sans intervention de scripts externes.

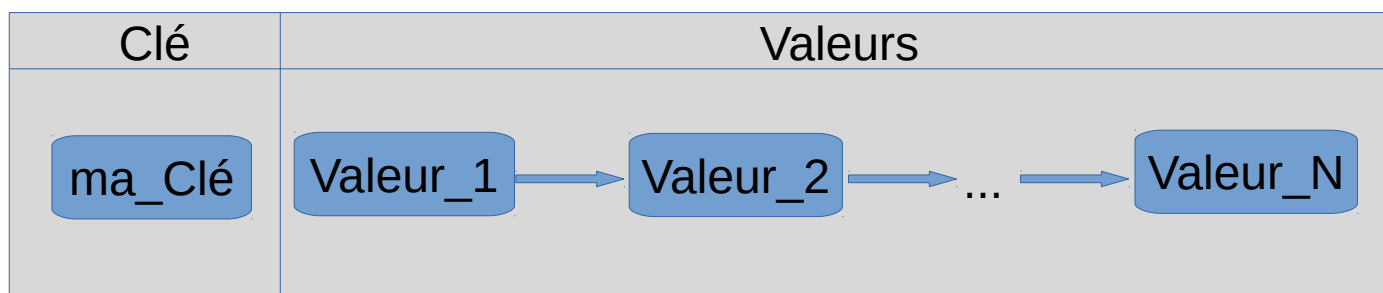
```
> set clé valeur
ok
```

```
> expire clé 1 (exprimé en seconde)
ok
> set clé valeur ex 10 (argument à l'insertion pour rajouté un délai
d'expiration)
> ttl clé (permet de connaître le temps restant à la clé avant destruction)
```

PEXPIRE, PTTL permettent les même actions mais exprimées en millisecondes.

c) Redis list

Redis utilise un système de liste chaînée pour la gestion des données un peu particulières, ceci afin de gérer la restructuration des données plus facilement. Ceci vient très certainement du fait que les données sont gérés en RAM donc très rapidement.



- **LPUSH** : rajoute un élément en début de liste
- **RPUSH** : rajoute un élément en fin de liste
- **LRANGE** : permet une extraction des données avec un certain indice

2 arguments sont à rajouter (le premier et le dernier élément)

→ A noter qu'on peut fournir un nombre négatif ainsi pour aller du premier au dernier élément :

```
> LRANGE clé 0 -1
```

- **RPOP** et **LPOP** permettent de retirer le dernier ou premier élément de la liste et de le retourner en résultat
 - retourne « *nil* » si plus de valeurs
- **LTRIM** : A l'instar de **LRANGE** permet une extraction d'une partie de la liste avec un indice de début et de fin + suppression des valeurs en dehors de la liste définie.
 - Utilisé surtout pour un stockage de messages ou de logs par exemple où l'on a besoin des derniers messages, le reste pouvant être supprimé.
- **BRPOP** et **BLPOP** : fonctions équivalentes à RPOP ou LPOP à part qu'elles ne retournent pas de résultat s'il n'y a plus de valeurs.
 - il est possible de définir un délai de réponse en secondes
 - BRPOP clé 5 (pour 5 secondes)
- **LLen** : donne la longueur d'une liste

Particularité : lorsqu'une liste est vide, Redis supprimera automatiquement la clé associée.

d) Redis hashes

Le hash de Redis associe une clé à un ensemble de clé-valeur

Clé	Valeurs	
ma_Clé	Clés	Valeurs
	sous-clé_1	« chaine1 »
	sous-clé_2	« chaine2 »
	sous-clé_3	« chaine3 »

Exemple :

```
> hmset user:1000 username antirez birthyear 1977 verified 1
OK
> hget user:1000 username
"antirez"
> hget user:1000 birthyear
"1977"
> hgetall user:1000
1) "username"
2) "antirez"
3) "birthyear"
4) "1977"
5) "verified"
6) "1"
```

Commandes :

HMSET : permet d'insérer dans une table de Hachage (crée la table si elle n'existe pas)

HGET : permet d'obtenir la valeur d'une clé dans une table de hachage

HGETALL : renvoie une liste composée de la clé puis la valeur jusqu'à ce qu'il n'y ait plus de couple de clé-valeur

HINCRBY : permet d'incrémenter une valeur dans une table de hashage pour une clé donnée

```
> hincrby user:1000 birthyear 10
(integer) 1987
> hincrby user:1000 birthyear 10
(integer) 1997
```

liste des commandes sur les tables de hachage : <https://redis.io/commands#hash>

e) Redis sets (ensembles non triés)

Il s'agit d'un stockage d'ensemble de string non ordonné appartenant à un ensemble.

SADD : permet d'ajouter un élément à un ensemble

```
> SADD ensemble1 chaine1 chaine2 chaine3
```

MEMBER : renvoie 1 ou 0 si une chaîne appartient à un ensemble. Cette fonctionnalité permettra par exemple de stocker des objets n'ayant pas d'ordre entre eux

SINTER : permet de calculer l'intersection de plusieurs ensembles (réunion des résultats)

SPOP : permet de retirer un nombre défini de résultats dans un ensemble, aléatoirement puisque l'ensemble est non ordonné

SUNIONSTORE : permet de faire une union de plusieurs listes et en crée une nouvelle

SCARD : renvoie le nombre d'élément d'un ensemble

f) Redis sorted sets (ensembles triés)

Il s'agit de stocker un ensemble de valeurs définie avec un score pour chaque valeur, le score sera ordonné du plus petit au plus grand.

Clé	Valeurs	
ma_Clé	scores	Valeurs
	1	« chaine1 »
	50	« chaine2 »
	70	« chaine3 »

Exemple :

```
> ZADD ensemble 1 chaine1
> ZADD ensemble 2 chaine2
> ZADD ensemble 3 chaine1
> ZRANGE ensemble 0 10 WITHSCORES (le dernier argument permet d'afficher le
score et non obligatoire)
1) « chaine1 »
2) « 1 »
3) « chaine2 »
4) « 2 »
5) « chaine1 »
6) « 3 »
```

→ le score étant unique pour une valeur donnée, il n'y aura pas de doublon.

Cette structure est stockée sous forme de hash dans Redis, la recherche par score est donc très rapide.

L'insertion d'un élément est $O(\log(N))$ contrairement aux ensembles non ordonnés, mais offre la possibilité de recherche sur un score en particulier.

ZREVRANGE : permet d'avoir une liste inversée du plus grand au plus petit.

```
> ZREVRANGE ensemble 0 -1
```

Des opérateurs existent pour avoir par exemple

→ Un sous ensemble ordonné borné entre 2 scores, inférieur ou supérieur.

→ D'avoir le score, pour une chaîne donnée.

→ Redis offre la possibilité de calculer l'intersection ou l'union de deux ensembles.

g) Redis Hyperloglogs

Cette structure de données provient de l'algorithme Hyperloglog qui est une approche probabiliste d'apparition d'éléments dans un ensemble de données. Il offre de bonnes performances et un stockage très réduit pour les calculs même si la base contient des millions d'entrées. Il s'agit de calculer une approximation du nombre d'occurrences d'un élément dans un ensemble avec un marge d'erreur très acceptable.

V. Base de données géospatiale

Redis propose plusieurs fonctions pour la gestion des bases de données géospatiales. Les valeurs géospatiales sont sous la forme (x,y) associées à un nom où x est la longitude et y la latitude.

Les coordonnées géospatiales sous Redis sont stockées dans un ensemble trié comme décrit préalablement afin d'améliorer les performances de recherche.

Les recherches et le score pour l'insertion d'une coordonnée se réalise par la « **formule de Haversine** » qui permet d'établir une distance dans une sphère à partir d'une longitude et latitude. La formule est expliquée sur wikipedia (https://fr.wikipedia.org/wiki/Formule_de_haversine) mais dépasse le cadre de cette étude.

Sur la documentation officielle de Redis, il est dit que la terre n'étant pas tout à fait sphérique, l'erreur maximum avec cette technique n'est que de 0,5 % dans le pire des cas, ce qui est tout à fait acceptable.

Il n'existe pas d'opérateur spécifique pour la suppression d'une coordonnée (GEODEL) mais étant stockée dans un ensemble trié, il est possible d'utiliser ZREM associé aux ensembles triés de Redis.

Les valeurs autorisés pour les coordonnées sont :

- Longitudes : -180 à 180 degrés
- Latitudes : -85.05112878 à 85.05112878 degrés

Redis interdira des valeurs au-delà de ses limites

1) Fonctions d'insertion et de recherche

a) GEOADD

Cet opérateur sert à ajouter des coordonnées dans un ensemble nommé représenté sous forme d'item.

Un item est un ensemble de coordonnées défini par un nom que l'on peut représenter : nom(x,y)

```
GEOADD <nom_item> <longitude> <latitude> <nom_coordonnée>
```

On peut également ajouter un ensemble de coordonnées dans un item :

```
GEOADD <nom_item> <longitude1> <latitude1> <nom_coordonnée1> ... <longitudeN>  
<latitudeN> <nom_coordonnéeN>
```

Complexité : $O(\log(N))$ à l'insertion où N est le nombre d'éléments

b) GEOHASH

Pour le stockage de coordonnées géospatiales, Redis utilise une fonction se nommant « geohash » qui est un algorithme public de géocodage de coordonnées géographiques (longitude/latitude). Ainsi, une coordonnée peut être stockée sous la forme d'une chaîne de caractères de 52 bits avec un taux d'erreur extrêmement faible.

Cet algorithme réduit nettement l'espace de stockage nécessaire pour une coordonnée.

La fonction « GEOHASH » retourne une ou plusieurs coordonnées sous la forme de cette chaîne de caractère de 11 caractères (52 bits).

Cette chaîne de caractère n'est pas directement exploitable, il faudra la décoder par exemple avec le site <http://geohash.org> qui a été créé pour l'exploitation de cette donnée.

L'algorithme est expliqué sur le lien wikipedia : « <https://fr.wikipedia.org/wiki/Geohash> » mais dépasse le cadre de cette étude.

```
GEOHASH <nom_item> <nom_coordonnée1> ... <nom_coordonnéeN>
```

retourne un tableau de chaîne de 11 caractères qui devront être décodés

```
1) « sqc8b49rny0 »
```

```
...
```

```
N) « 11 caractères »
```

complexité : $O(\log(N))$ pour chaque item recherché

c) GEOPOS

De même, cette fonction retourne cette fois les coordonnées (longitude et latitude) stockées.

Cette fonction renvoie un tableau de coordonnées (longitude, latitude) d'un ou plusieurs éléments en argument d'un item.

```
GEOPOS <nom_item> <nom_coordonnée>
```

```
1) <longitude_de_nom_coordonnée>
```

```
2) <latitude_de_nom_coordonnée>
```

```
GEOPOS <Nom_item> <nom_coordonnée1> ... <nom_coordonnéeN>
```

```
1)
```

```
1) <longitude_de_nom_coordonnée1>
```

```
2) <latitude_de_nom_coordonnée1>
```

```
...
```

```
N)
```

```
1) <longitude_de_nom_coordonnéeN>
```

```
2) <latitude_de_nom_coordonnéeN>
```

Complexité : $O(\log(N))$

2) Opérateur de comparaison

Redis propose 2 opérateurs pour comparer ces données, la base de données géospatiale étant stockée dans un ensemble trié, ces opérateurs offrent de très bonnes performances mais reste limités dans leur utilisation. On ne travaillera que sur des coordonnées « statiques » positionnées autour de la terre.

- Distance entre deux éléments

- Les éléments les plus proches dans un rayon donné

Par exemple, dans la navigation, il pourra être important de connaître la distance entre 2 bateaux à un instant T de manière rapide.

a) GEODIST

Cet opérateur permet de calculer des distances entre deux coordonnées, par défaut en mètre mais il est possible de rajouter un argument pour déterminer la métrique :

- m : mètre
- km : kilomètre
- mi : miles
- ft : pieds

```
GEODIST <nom_item> <nom_coordonnée1> <nom_coordonnée2> km
```

Les distances ne peuvent être calculées qu'à l'intérieur d'un même item car il s'agit d'ensemble trié. Il ne sera pas possible de faire de calcul entre deux coordonnées provenant de deux items différents.

Complexité : $O(\log(N))$

b) GEORADIUS

GEORADIUS key longitude latitude radius m|km|ft|mi [WITHCOORD] [WITHDIST] [WITHHASH] [COUNT count] [ASC|DESC] [STORE key] [STOREDIST key]

Cette opérateur retourne toutes les coordonnées autour d'un point et d'un rayon donnés en argument.

Plusieurs options de retour existent sur cet opérateur.

- WITHDIST : retourne les distances entre le point origine et les points trouvés par calcul
- WITHCOORD : retourne les coordonnées des points trouvés
- WITHHASH : retourne le hash des points trouvés
- COUNT : retourne un nombre donné de points les plus proches
- STORE et STOREDIST : permet de stocker les résultats avec une clé donnée

Il est possible de demander le tri des points retournés en fonction de la distance par rapport au point d'origine.

- ASC : tri par ordre croissant des distances
- DESC : tris par ordre décroissant des distances

```
GEORADIUS <nom_item> <longitude> <latitude> <distance> <métrique> (WITHDIST)
(WITHCOORD) (WITHHASH) (ASC ou DESC) [COUNT count] [STORE key] [STOREDIST key]
1)
  1) <nom_élément>
  2) <distance>
  3)
    1) <longitude>
    2) <latitude>
```

```
...
N)
  1) <nom_élément>
  2) <distance>
  3)
    1) <longitude>
    2) <latitude>
```

Complexité : $O(N + \log(M))$: N étant le nombre d'éléments retournés dans un rayon donné et M étant le nombre d'éléments de l'item.

c) GEORADIUSBYMEMBER

GEORADIUSBYMEMBER <nom_item> <nom_membre> <distance> <métrique> (WITHDIST) (WITHCOORD) (WITHHASH) (ASC ou DESC) [COUNT count] [STORE key] [STOREDIST key]

Fonctionne de la même manière que GEORADIUS à part que le point à comparer fait déjà partie de la base d'index géospatiale

Complexité : $O(N + \log(M))$: N étant le nombre d'éléments retournés dans un rayon donné et M étant le nombre d'éléments de l'item.

VI. Le système de sauvegardes

Le problème des bases de données fonctionnant sur RAM est la persistance des données. Il existe un système de réplication et de partitionnement de données relativement robuste, mais en cas d'extinction de serveurs, les données en ram sont effacées. Redis propose donc un système de sauvegarde sur disque pour davantage de robustesse en cas de crash ou de changement de version principalement.

1) Créer une copie exacte d'un serveur

Un moyen simple de sauvegarder des données est de créer un serveur de réplication.

Redis facilite cet usage par un mécanisme automatisé. Ceci consiste à créer une exacte copie d'un serveur sur un autre, avec synchronisation automatique des données par un fonctionnement maître esclave. On rajoutera donc un serveur esclave à un autre qui sera considéré comme maître où les lectures seules seront autorisées par défaut, les écritures étant l'exclusivité du serveur maître.

Il faudra pour cela un serveur créé avec Docker par exemple, et se connecter avec le client Redis pour lui dire de se connecter en esclave sur un autre serveur.

```
REPLICAOF <host_name> <port>
REPLICAOF NO ONE → Pour stopper la réplication
```

Le fonctionnement est simple, une fois un serveur connecté en esclave, le maître envoie une image du contenu de sa base. Une fois ceci fait, à chaque mise à jour de ses données, le maître enverra ces mises à jour à l'esclave sous forme de dataset.

Pour des raisons de performances, redis utilise par défaut un système de réplication asynchrone, donc sans acquittement obligatoire du serveur distant.

Si le maître ne répond plus, l'esclave se met en attente de reconnexion et resynchronisera uniquement si le maître est de nouveau actif, en autorisant toujours les lectures et interdisant les écritures.

Le fonctionnement par défaut de Redis peut être modifié dans le fichier de configuration redis.conf. Il est possible de désactiver le read-only, ou de se mettre en réplication synchrone par exemple.

Il est également possible de définir des règles d'acquittements d'écriture sur N replica

```
min-replicas-to-write <nombre de réplica>
min-replicas-max-lag <nombre de secondes>
```

Vérifier les informations de réplication d'un serveur

et pour obtenir des informations sur la réplication : info replication

```
ROLE → pour obtenir des informations sur le master quand on l'exécute depuis un esclave.
```

2) Persistance des données sur disques

Redis propose différentes options de persistance des données qui sont représentés par 2 fichiers bien distincts :

- RDB pour Redis Database Backup, il s'agit d'une image des données présentes dans une base de données appelé aussi « snapshot ».
- AOF pour Append Only File, il s'agit d'une journalisation des modifications effectuées sur la base.

a) RDB

Redis propose la mise en place de snapshot manuellement avec le client en ligne. Ceci permet de réaliser des tests.

Pour sauvegarder le contenu d'une base enregistrée en ligne de commande sur le serveur où est connecté le client

```
BGSAVE
```

Le client permet de modifier le répertoire de sauvegarde :

```
CONFIG GET dir
CONFIG SET dir <répertoire> pour en changer le répertoire de sauvegarde
```

La base sera sauvegardée dans le fichier dump.rdb

Sauvegarde automatisée

Dans son fichier de configuration Redis propose des sauvegardes automatisées toutes les X secondes en fonction du nombre de modifications et la possibilité d'imbriquer ces règles.

Exemple de sauvegarde toutes les 300 secondes (environ 5 minutes) si au moins 10 clés ont changé.

```
save 300 10
```

Cette technique comporte des avantages qui sont une compression des données et une rapidité d'extraction si une base est à reconstruire. Par contre elle comporte des désavantages non négligeables, il s'agit d'une technique très consommatrice en temps processeur et plus ou moins longue à exécuter en fonction du nombre de données stockées. En cas de crash serveur ou de reboot,

les données sont perdues et la reconstruction se fera avec perte de données entre le laps de temps où la sauvegarde a été faite et le moment de la perte de données.

b) AOF

Cette technique vise à journaliser toutes les commandes de modification et d'insertion apportées sur une base, mais peut être volumineuse à stocker. Par contre cette technique est beaucoup plus rapide à écrire que la précédente, il conviendra donc de coupler les deux techniques de persistance afin d'avoir un maximum de sécurité en cas de perte de données.

En cas de perte de données, il suffira de dérouler les commandes stockées dans ce fichier et retrouver les données.

En cas de reconstruction de base, cette technique est beaucoup moins efficace que le fichier RDB.

Plusieurs paramètres existent, la journalisation permanente (à chaque modification) et la journalisation toutes les X secondes, Redis recommande une journalisation toutes les secondes qui semble être un compromis entre efficacité et consommation processeur.

c) Stratégie de sauvegarde

Il ne faut pas oublier qu'un reboot serveur cause la perte des données stockées en mémoire RAM (mémoire volatile).

Il est recommandé de cumuler les 2 techniques de sauvegardes simultanément, la première est un snapshot de la base toutes les X minutes avec de la compression, voire une fois par jour pour les bases de données conséquentes. Cette technique comporte tout de même des risques non négligeables de perte de données entre la sauvegarde et le crash potentiel. Le fichier AOF permettra d'enregistrer les commandes de modification et d'insertion qu'il sera possible d'agréger au snapshot de la base.

VII. Fonctionnement du mode cluster

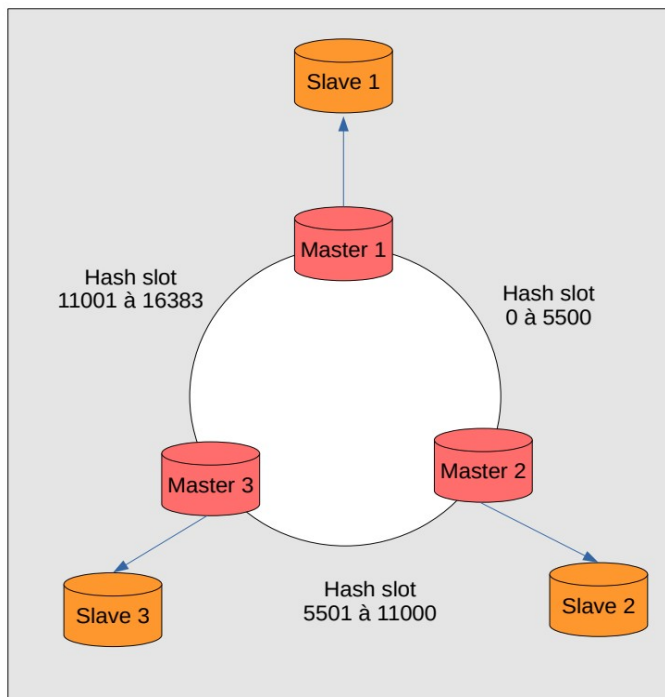
Comme beaucoup de bases NoSQL, Redis dispose d'un mode cluster afin d'avoir une certaine tolérance aux pannes.

Ce mode de fonctionnement fonctionne en maître/esclave, où le maître reçoit toutes les écritures et les envoie aux esclaves en mode asynchrone avec en option une confirmation par l'esclave.

Dans cet exemple, nous allons mettre en place le mode cluster de Redis minimum qui propose 3 serveurs maîtres se partageant les données sur la clé, et un esclave sur chacun qui est une réplication du maître. Ceci est la configuration minimum du mode cluster que peut proposer Redis, il sera possible de rajouter des maîtres et des esclaves si les besoins se font sentir.

Les données sous Redis sont réparties selon la clé dans une table de hachage de 16384 slots. Cette table de hachage n'est pas évolutive mais semble relativement importante pour une bonne distribution des données.

Image du mode cluster minimum de Redis (représentation du hash ring sous Redis)



1) Mise en place :

Tout d'abord, il faudra lancer 6 serveurs Redis, si en local comme dans mon exemple, il faudra définir un port différent sur chaque serveur avec ces commandes actives dans le fichier de configuration :

```
port 7000 → Pour définir le port d'écoute (à adapter sur chaque serveur)
cluster-enabled yes → Activer le mode cluster
cluster-config-file nodes.conf → fichier du cluster avec table de routage et tous les esclaves.
Cluster-node-timeout 5000 → délai d'inactivité acceptable
appendonly yes → activer les logs du fichier AOF
```

Par défaut, la répartition des clés se fait comme sur le dessin précédent.

Le cluster se lance avec le client en ligne de commande :

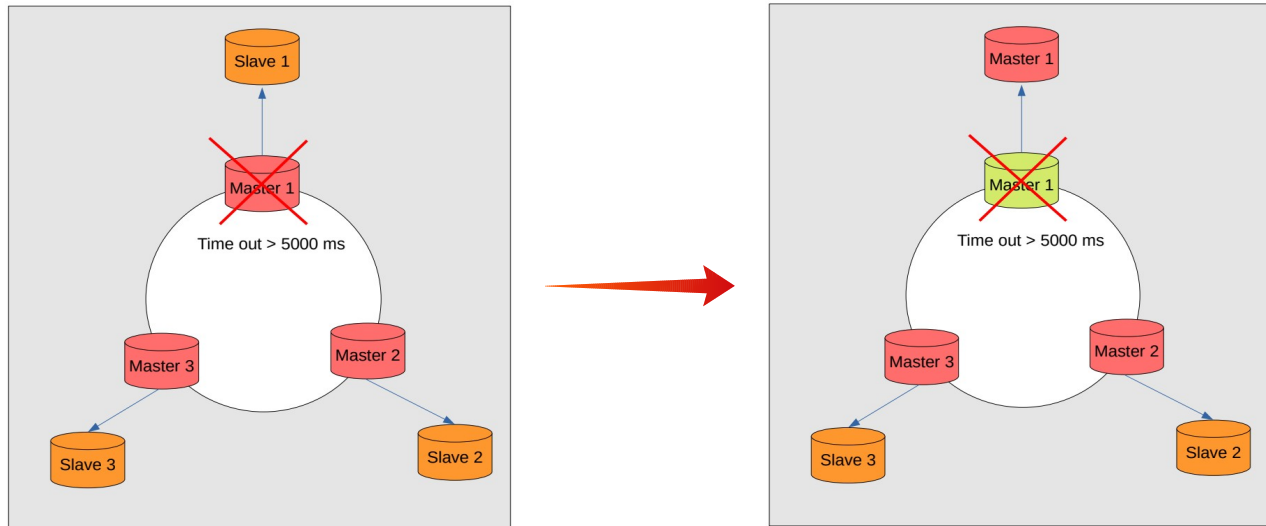
```
redis-cli --cluster create 127.0.0.1:7000 127.0.0.1:7001 127.0.0.1:7002
127.0.0.1:7003 127.0.0.1:7004 127.0.0.1:7005 --cluster-replicas 1
```

Si on veut 2 esclaves par maître, il faudra 9 serveurs et un cluster-replica à 2

2) En cas de panne :

En cas de panne d'un des master par exemple, il y a négociation avec les autres master pour refaire la table de routage, l'esclave devient le nouveau maître au bout d'un délai défini dans le fichier de configuration.

Schéma montrant l'indisponibilité d'un des master et l'élection d'un nouveau master



Si le master revient en état, il synchronise ses données avec le nouveau master et devient esclave à son tour.

Le fichier node.conf

Ce fichier fait office de table de routage et est dupliqué sur l'ensemble des serveurs du cluster. Ce fichier reste lisible et compréhensible mais ne doit pas être modifié manuellement

Le master et l'esclave sont hors ligne

Dans ce cas, le fonctionnement par défaut est de stopper le cluster en attendant que la situation soit rétablie et que tous les masters répondent.

Redis fonctionnera seulement et seulement si toutes les données sont accessibles, donc si on a au minimum tous les masters opérationnels.

En cas de perte des données et reprise sur panne

Dans le cas où un master et son esclave sont éteints, toutes les données sont perdues. Lors de la reconnexion, le maître ou élu maître suite à reconnexion vérifiera ses sauvegardes et les utilisera, dans un premier temps, il fera une restauration avec le fichier rdb puis en complément utilisera le fichier AOF, ces 2 mécanismes ont été décrits préalablement.

Redistribution des requêtes

Seuls les masters sont capables d'écrire dans la base, puis dupliquent l'information au(x) esclave(s), si une demande d'écriture ou de lecture est demandée au mauvais serveur, celui-ci renvoie la requête sur le bon serveur.

On se connecte à un des serveurs avec le clients en ligne en mode cluster (argument c) :

```
redis-cli -c -p 7000
redis 127.0.0.1:7000> set foo bar → insertion de clé/valeur simple
-> Redirected to slot [12182] located at 127.0.0.1:7002 → redirigé sur le serveur
concerné
OK
redis 127.0.0.1:7002> set hello world → insertion d'un autre couple clé/valeur
-> Redirected to slot [866] located at 127.0.0.1:7000 → redirigé sur le serveur
concerné
OK
redis 127.0.0.1:7000> get foo
-> Redirected to slot [12182] located at 127.0.0.1:7002
```

```
"bar"
redis 127.0.0.1:7000> get hello
-> Redirected to slot [866] located at 127.0.0.1:7000
"world"
```

On remarque ici que les requêtes sont redirigées sur les serveurs concernés.

3) Répartition des clés ou resharding

Redis propose sur son client une technique de répartition de clés sur les différents serveurs du cluster. Basiquement, Redis propose de déplacer un certain nombre de slots qu'on lui définit manuellement sur différents serveurs. Cela se nomme le resharding, pour lancer la commande :

```
redis-cli --cluster reshard 127.0.0.1:7000
```

Vient ensuite une question :

```
How many slots do you want to move (from 1 to 16384)?
```

Une fois renseigné le nombre de slots que l'on souhaite transférer, Redis demande l'id sur lequel transférer les slots. Redis prend les premier slots et les transfère avec un processus long, puis met à jour la table des partitions sur l'ensemble des serveurs.

VIII. Clients Redis pour les langages de programmation

Pour les exemples de cette partie, nous allons décrire quelques fonctionnalités intéressantes du client sur Python. Tous les autres clients fonctionneront sur le même principe.

Plusieurs modules de clients existent sous python, en voici deux intéressants qui me semblent à retenir.

Redis-py qui est le plus évolué et l'un des modules les mieux maintenu.

redis-py-cluster qui est un fork (à priori) de redis-py, c'est celui-ci que j'ai surtout utilisé tant il prend en charge correctement le mode cluster proposé par Redis.

1) Utilisation de la bibliothèque

Il faudra installer le module redis-py-cluster avec la commande « pip install ».

La connexion et l'envoi de données restent très simplifié, en voici un exemple :

```
from rediscluster import RedisCluster

# requiert au moins un serveur connecté au cluster, la bibliothèque fait le
# reste pour importer toute la configuration du cluster
startup_nodes = [{"host": "127.0.0.1", "port": "7000"}]
rc = RedisCluster(startup_nodes=startup_nodes, decode_responses=True)

# requêtes d'ajout et récupération de données.
rc.set('clé', 'valeur')
rc.get('clé')
```

A priori, toutes les commandes de requêtes que l'on peut trouver sur le langage d'interrogation de Redis-client sont présentes.

2) Mode pipeline de Redis

Une fonction de pipeline existe sous Redis, qui revient à stocker des requêtes avec un acquittement groupé, ceci permet de réduire énormément la latence, surtout dans les cas d'insertion massives de données.

Cette technique est appropriée dans les cas d'insertion massives de données.

Voici un exemple de code illustratif qui vise à introduire 1000 clés avec une valeur quelconque, qui devraient être bien réparties dans les clusters.

```
from rediscluster import RedisCluster

# requiert au moins un serveur connecté au cluster, la bibliothèque fait le
# reste pour importer toute la configuration du cluster
startup_nodes = [{"host": "127.0.0.1", "port": "7000"}]
rc = RedisCluster(startup_nodes=startup_nodes, decode_responses=True)

pipe = rc.pipeline()

for i in range(1000):
    pipe.set(f"key:{i}", f"test_{i}")
pipe.execute()
```

L'exécution du code se réalise en quelques millièmes de secondes car chaque instruction est exécutée indépendamment dans un thread sans acquittement attendu.

Voici une commande pour vérifier les clés avec le client :

```
redis-cli --cluster check 127.0.0.1:7000
127.0.0.1:7000 (33cecf78...) -> 341 keys | 5461 slots | 1 slaves.
127.0.0.1:7001 (4f654aee...) -> 323 keys | 5462 slots | 1 slaves.
127.0.0.1:7002 (6953ec2c...) -> 336 keys | 5461 slots | 1 slaves.
```

Nous pouvons voir que les clés ont bien été réparties entre les serveurs et toutes insérées.

IX. Mise en application dans un jeu de données

Pour le fonctionnement du mode cluster, les tests ont été réalisés avec le prix de près de 800 cryptomonnaies en temps réels, les données arrivent sous forme de trame Json qu'il faudra formater pour correspondre à la structure clé/valeur de Redis.

Près de 800 trames correspondant à chaque paire de cryptomonnaies sont reçues toutes les minutes et insérés dans Redis à l'aide du fonctionnement de la méthode « pipeline »

exemple de trame reçue : {'tickers' : 'MKRBNB', 'price': 0.125986, 'time' : '1596121131774'}

Avant d'envoyer cette chaîne à Redis, il a fallu réfléchir au format de la clé et au format de la valeur correspondant à la clé, ainsi qu'à la structure la plus adaptée.

Le but étant de trouver une clé qui soit la mieux répartie dans le scope de la fonction de hachage, je suis parti sur une clé prenant en compte une cryptomonnaie et un horodatage, par exemple :

'MKRBNB:20200720' puisque le but est de conserver des valeurs sur une journée.

La valeur est une trame Json avec le prix et l'horodatage précis :

{'price': 0.125986, 'time' : '1596121131774'}

A des fins d'analyse du cours sur une journée dans une paire de cryptomonnaies, la structure la plus adaptée restait la liste chaînée expliquée dans les structures.

1) Code python

```
import time
from binance.client import Client # Import the Binance Client
from rediscluster import RedisCluster
import json

# import des clés d'API du broker Binance qui sont masquées
PUBLIC = '*****'
SECRET = '*****'

# Instanciation du Client Binance
client = Client(api_key=PUBLIC, api_secret=SECRET)

# Instanciation de la connexion redis
startup_nodes = [{"host": "127.0.0.1", "port": "7000"}]
rc = RedisCluster(startup_nodes=startup_nodes, decode_responses=True)

# Création du pipeline
pipe = rc.pipeline()

# la fonction de récupération des données fonctionnera en permanence
while True:
    # récupération des données
    tickers = client.get_all_tickers()
    time_res = client.get_server_time()
    ts = int(time_res['serverTime']/1000)

    # formatage des données
    for ticker in tickers:
        value = {
            'time': time_res['serverTime'],
            'price': ticker['price']
        }
        value_string = json.dumps(value)
        key = f"{ticker['symbol']}:{time.strftime('%Y%m%d',
time.localtime(ts))}"

        # données mises dans le pipe
        pipe.rpush(key, value_string)
    pipe.execute() # Exécution du pipe
    time.sleep(60) # Récupération des valeurs toutes les minutes
```

Ce code relativement simple se connecte à l'API de Binance et récupère toutes les paires de valeurs de cryptomonnaie avec le prix à un instant t, formate les données et les envoient sur Redis.

Au bout de 1h, 893 paires de clés insérés comportant chacun 60 trames JSON avec le prix et l'horodatage précis.

Voici la répartition des clés :

```
redis-cli --cluster check 127.0.0.1:7000
127.0.0.1:7000 (33cecf78...) -> 309 keys | 5461 slots | 1 slaves.
127.0.0.1:7001 (4f654aee...) -> 303 keys | 5462 slots | 1 slaves.
127.0.0.1:7002 (6953ec2c...) -> 281 keys | 5461 slots | 1 slaves.
```

2) Récupération des valeurs

Dans notre cas, il est nécessaire de récupérer toutes les valeurs d'une paire de cryptomonnaies sur une journée entière.

Voici la fonction de récupération qui renvoie un tableau exploitable par python :

```
LRANGE 'MKRBNB:20200720' 0 -1
```

On récupère la liste des prix de la paire MKR-BNB sur la journée du 20-07-2020.

X. Conclusion

Redis est une base de données en croissance continue. Restant simple dans son utilisation et sa structure de données mais puissante dans sa richesse de stockage, elle fait partie des bases de données les plus performantes du marché actuelle.

Redis s'adapte très bien aux tendances actuelles du cloud, car facilement scalable dans son mode cluster. Il faut cependant faire attention car fonctionnant en RAM, le prix du stockage est non négligeable, Redis restera donc une solution pour les applications ou partie d'applications ayant des besoins de rapidité au stockage limitée.

Ne fonctionnant pas de la même manière que la plupart des bases de données NoSQL qui travaillent avec des trames JSON ou équivalent, il faudra réfléchir à une structuration des données 'clés/valeurs' adaptée à Redis.

Aucun contrôle de données n'a été présenté car Redis laisse la responsabilité de ce point aux concepteurs de l'application. Ce dernier point doit être étudié avec soin sous contrainte d'avoir une base de données inexploitable.

XI. Références bibliographique

- *Documentation officielle de Redis*, <https://redis.io/documentation>
- Stack Overflow contributors. (2020). *Learning Redis* [PDF file]. Available from <https://riptutorial.com/ebook/redis>
- [Haber](#), I. H. (2014, 11 Septembre), *So, You're Looking for the Redis GUI?*, <https://redislabs.com/blog/so-youre-looking-for-the-redis-gui/>
- Wikimedia, *Hyperloglog - HyperLogLog*, <https://fr.qwe.wiki/wiki/HyperLogLog>
- Formule de haversine. (2020, avril 19). Wikipédia, *l'encyclopédie libre*. Page consultée le 11:39, avril 19, 2020 à partir de http://fr.wikipedia.org/w/index.php?title=Formule_de_haversine&oldid=169751490.
- Geohash. (2020, janvier 5). Wikipédia, *l'encyclopédie libre*. Page consultée le 20:26, janvier 5, 2020 à partir de <http://fr.wikipedia.org/w/index.php?title=Geohash&oldid=166075672>.
- Grokzen (2020). Github, *Redis-py-cluster*, <https://github.com/Grokzen/redis-py-cluster>